

Casting Defect Detection Using Deep Learning (CNN & Transfer Learning)

A Comparative Study of Custom CNN and MobileNetV2 for Automated Quality Control

INTRODUCTION

1.1 Background

Quality control is a critical aspect of manufacturing, particularly in the casting industry where defects can lead to product failure, customer dissatisfaction, and significant financial losses. Traditionally, casting defects are identified through manual visual inspection, which is:

- Time-consuming – Large volumes of products need inspection.
- Subjective – Human fatigue and inconsistency affect accuracy.
- Costly – Requires skilled labor and continuous training.

1.2 Problem Statement

The goal of this project is to develop an automated system for detecting casting defects using deep learning techniques. The system classifies images into two categories:

Class	Description
ok_front	Non-defective product
def_front	Defective product

1.3 Objectives

1. Build a Custom CNN from scratch for binary classification.
2. Implement Transfer Learning using MobileNetV2 (pre-trained on ImageNet).
3. Compare the performance of both models.
4. Develop a deployable solution for industrial quality control.

2. DATASET DESCRIPTION

2.1 Source

The dataset is sourced from Kaggle: **Real-life Industrial Dataset of Casting Product**.

2.2 Dataset Statistics

Class	Training Images	Test Images	Total
ok_front	2,875	262	3,137
def_front	3,758	453	4,211
Total	6,633	715	7,348

2.3 Image Specifications

- **Size:** 224×224 pixels (resized for model input)
- **Channels:** RGB (3 channels)
- **Format:** JPEG images

2.4 Class Distribution

The dataset has a slight class imbalance:

- Defective images = 56.7% of training data
- Non-defective images = 43.3% of training data

METHODOLOGY

3.1 Data Preprocessing

3.1.1 Image Resizing

All images were resized to 224×224 pixels to match the input requirements of both models.

3.1.2 Data Augmentation

To prevent overfitting and improve generalization, the following augmentations were applied to training data:

- Random Horizontal Flip – 50% probability
- Random Rotation – ± 5 degrees
- Random Zoom – 10%
- Random Contrast – $\pm 10\%$

3.1.3 Normalization

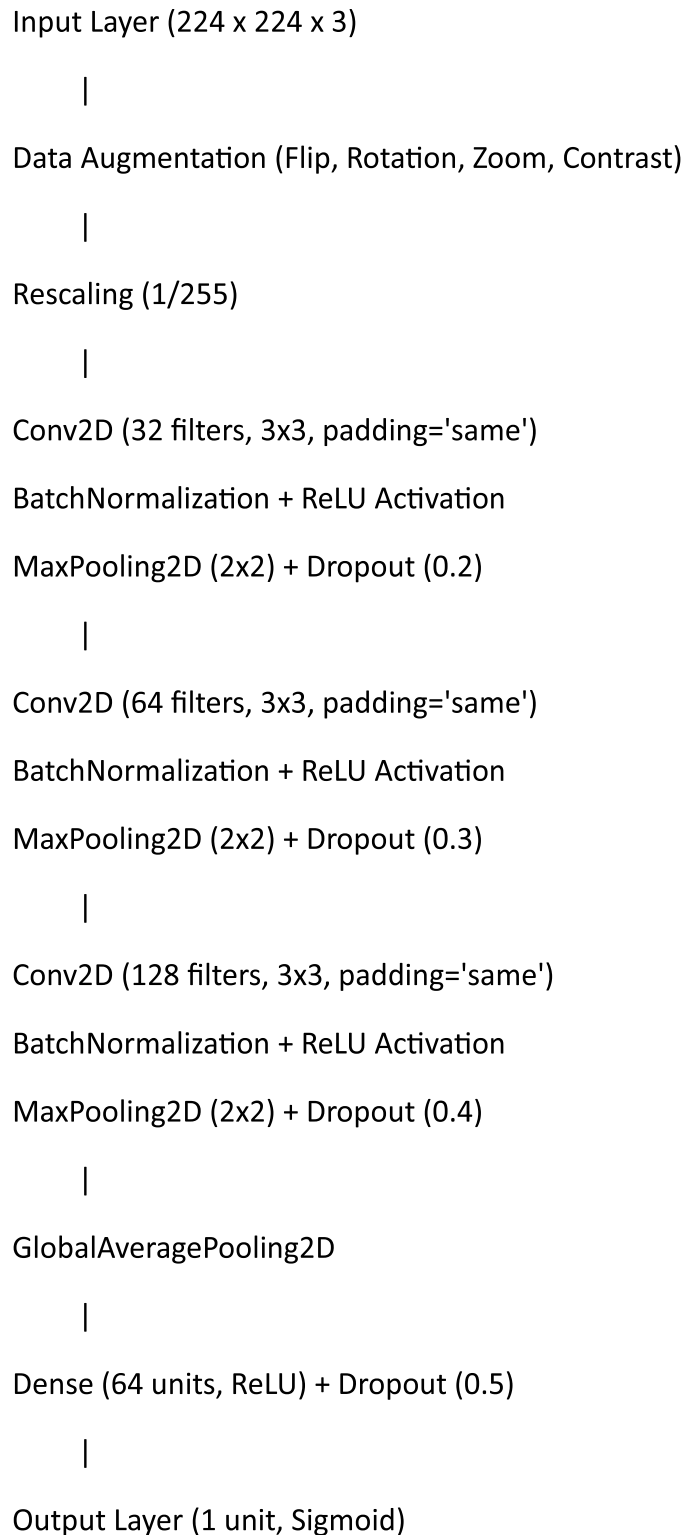
- **Custom CNN:** Pixel values normalized to $[0, 1]$ range.
- **MobileNetV2:** Pixel values normalized to $[-1, 1]$ range (as per MobileNetV2 requirements).

3.1.4 Train-Validation Split

- **Training:** 80% of training data (approx 5,306 images)
- **Validation:** 20% of training data (approx 1,327 images)
- **Test Set:** 715 images (separate, unseen data)

3.2 Model Architectures

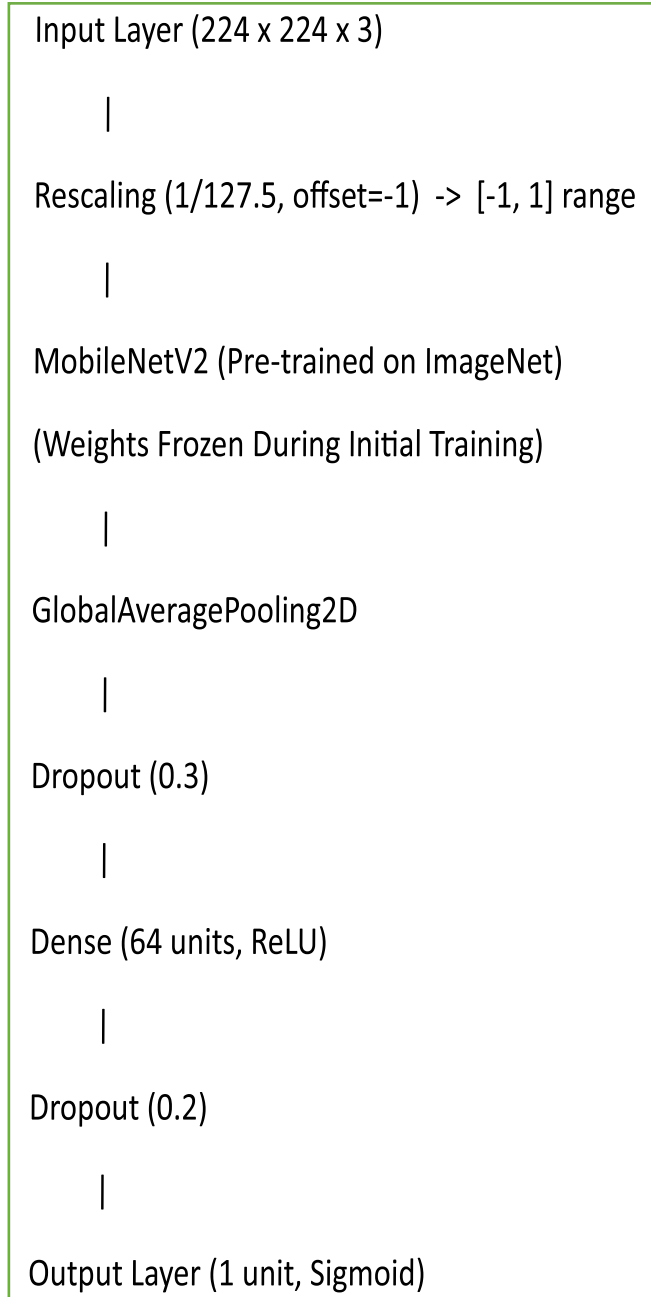
Model 1: Custom CNN (Built from Scratch)



Key Features:

- BatchNormalization – Accelerates training and improves stability.
 - Dropout – Reduces overfitting.
 - GlobalAveragePooling – Reduces parameter count and prevents overfitting.
-

Model 2: MobileNetV2 (Transfer Learning)



Key Features:

- Transfer Learning – Leverages pre-trained weights from ImageNet.
- Inverted Residuals – Efficient architecture for mobile/edge deployment.
- Faster Convergence – Requires fewer epochs due to pre-trained weights.

3.3 Training Configuration

Hyperparameter	Custom CNN	MobileNetV2
Optimizer	Adam (lr=1e-3)	Adam (lr=1e-4)
Loss Function	Binary Crossentropy	Binary Crossentropy
Epochs	25 (with early stopping)	10 (with early stopping)
Batch Size	32	32
Early Stopping	Patience = 5	Patience = 4
Reduce LR	Factor=0.5, Patience=3	Factor=0.5, Patience=2
Minimum LR	1e-7	1e-7

3.4 Evaluation Metrics

Metric	Description
Accuracy	Overall correctness: $(TP + TN) / \text{Total}$
Precision	$TP / (TP + FP)$ — How many predicted positives are correct
Recall	$TP / (TP + FN)$ — How many actual positives were found
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
AUC Score	Area Under ROC Curve — Measures classification performance

3.5 Confusion Matrix Structure

	Predicted: ok_front	Predicted: def_front
Actual: ok_front	True Negative (TN)	False Positive (FP)
Actual: def_front	False Negative (FN)	True Positive (TP)

4. RESULTS

4.1 Custom CNN Results

Confusion Matrix

	Predicted: ok_front	Predicted: def_front
Actual: ok_front	109	153
Actual: def_front	12	441

Interpretation:

- True Negatives (TN): 109 -> Non-defective correctly identified ✓
- False Positives (FP): 153 -> Non-defective wrongly marked as defective ✗
- False Negatives (FN): 12 -> Defective wrongly marked as non-defective ✗
- True Positives (TP): 441 -> Defective correctly identified ✓

Classification Report

Class	Precision	Recall	F1-Score	Support
ok_front	0.90	0.42	0.57	262
def_front	0.74	0.97	0.84	453
Accuracy			0.77	715
Macro Avg	0.82	0.69	0.71	715
Weighted Avg	0.80	0.77	0.74	715

- **Test Accuracy:** 76.9%
- **AUC Score:** 0.9326 (Excellent discrimination)

4.2 MobileNetV2 Results

Confusion Matrix

	Predicted: ok_front	Predicted: def_front
Actual: ok_front	260	2
Actual: def_front	13	440

Interpretation:

- True Negatives (TN): 260 -> Non-defective correctly identified ✓
- False Positives (FP): 2 -> Non-defective wrongly marked as defective X
- False Negatives (FN): 13 -> Defective wrongly marked as non-defective X
- True Positives (TP): 440 -> Defective correctly identified ✓

Classification Report

Class	Precision	Recall	F1-Score	Support
ok_front	0.95	0.99	0.97	262
def_front	1.00	0.97	0.98	453
Accuracy			0.98	715
Macro Avg	0.97	0.98	0.98	715
Weighted Avg	0.98	0.98	0.98	715

- **Test Accuracy:** 97.9%
- **AUC Score:** 0.9977 (Outstanding performance)

4.3 Model Comparison

Final Comparison Table

Model	Validation Accuracy	Test Accuracy	Validation Loss
Custom CNN	78.7%	76.9%	0.590
MobileNetV2	98.5%	97.9%	0.063

5. DISCUSSION

5.1 Performance Analysis

Custom CNN

- **Strengths:** Simple architecture, built from scratch, ~77% accuracy.
- **Weaknesses:**
 - High false positives (153 non-defective incorrectly flagged as defective).
 - Recall for ok_front is low (0.42), meaning many non-defective products are missed.
 - Overfitting evident from validation accuracy (78.7%) lagging behind training accuracy (~84%).

MobileNetV2

- **Strengths:**
 - Exceptional accuracy – 97.9% on test set.
 - Very low false positives – Only 2 non-defective images flagged incorrectly.
 - Excellent recall – 0.97 for defective, 0.99 for non-defective.
 - Balanced performance across both classes.
 - Fast training – Achieved high accuracy in just 10 epochs.
 - Robust generalization – Validation accuracy matches training accuracy.
- **Weaknesses:** Slightly higher false negatives (13 defective images missed) compared to Custom CNN (12).

5.2 Industrial Quality Control Implications

- **False Negatives (FN)** are most critical in manufacturing (defective products passing inspection).
 - Custom CNN: 12 FN out of 453 defective images = 2.6% miss rate.
 - MobileNetV2: 13 FN out of 453 defective images = 2.9% miss rate.
 - → Both models have low miss rates, ensuring high defect detection.
- **False Positives (FP)** cause non-defective products to be incorrectly rejected:
 - Custom CNN: 153 FP out of 262 non-defective images = 58.4% false rejection rate.
 - MobileNetV2: 2 FP out of 262 non-defective images = 0.8% false rejection rate.
 - → MobileNetV2 significantly reduces unnecessary re-inspection cost.

5.3 Why MobileNetV2 Performed Better

1. **Transfer Learning Advantage:**
 - Pre-trained on ImageNet with 14 million images.

- Learned rich feature representations (edges, textures, shapes).
- Requires less data to achieve high accuracy.

2. Efficient Architecture:

- Inverted residuals with linear bottlenecks.
- Depthwise separable convolutions reduce parameter count.
- Better feature extraction.

3. Faster Convergence:

- Reached high accuracy in just 10 epochs.
- Lower learning rate (1e-4) resulted in stable training.

4. Batch Size Optimization:

- Training with batch size 32 balanced speed and gradient stability.

6. CONCLUSION

6.1 Summary

This project successfully demonstrated automated defect detection for casting products using deep learning. Two models were implemented and compared:

Model	Architecture	Test Accuracy	Key Strength
Custom CNN	3 Conv layers + Dense layers	76.9%	Simple, interpretable, low FN
MobileNetV2	Pre-trained on ImageNet	97.9%	High accuracy, very low FP

6.2 Key Findings

1. MobileNetV2 significantly outperforms the Custom CNN, achieving:

- 97.9% test accuracy vs 76.9%.
- 0.8% false positive rate vs 58.4%.
- Excellent recall for both classes.

2. Both models effectively detect defective products:

- MobileNetV2: 97% recall (440/453)
- Custom CNN: 97% recall (441/453)

3. MobileNetV2 is production-ready:

- High accuracy.
- Fast inference.
- Low computational requirements (suitable for edge devices).
- Minimal false positives reduce operational costs.

6.3 Industrial Relevance

The MobileNetV2 model can be deployed in manufacturing plants for automated quality inspection:

- Reduces manual effort and human error.
- Increases inspection speed (can process thousands of images/min).
- Cost-effective – reduces waste and rework costs.
- Scalable – can be integrated with conveyor belt cameras.

6.4 Limitations

1. Custom CNN struggled with non-defective class, resulting in many false positives.
2. Dataset only includes two classes (ok/def). Real-world manufacturing may have multiple defect types.
3. Images were captured in controlled conditions; performance may degrade with varying lighting or camera angles.
4. Model was trained on frontal views only; defects may appear on other angles.

Area	Potential Improvement
Data Collection	Expand dataset with more defect types and real-world variations.
Model Architecture	Try EfficientNet, ResNet, or Vision Transformers.
Deployment	Build web application (Flask/Streamlit) or mobile app.
Edge Deployment	Convert model to TensorFlow Lite for edge devices.
Explainability	Use Grad-CAM to visualize which regions led to classification.

6.5 Future Work

Area	Potential Improvement
Multi-Classification	Classify defect types (e.g., cracks, blowholes, inclusions).
Active Learning	Continuously improve model with new labeled data.